```
// ABYSS eBay Scanner — même principe que le scanner du 14 octobre, mais sur eBay
// Chaque nouvelle annonce eBay sous ton prix max = 1 alerte Discord, une seule f
// Zéro dépendance : Node 18+ suffit (Termux : pkg install nodejs).

import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";

const DIR = path.dirname(fileURLToPath(import.meta.url));
const SEEN_FILE = path.join(DIR, "data", "seen.json");

// ---------- .env (lu directement dans le dossier, plus besoin de .bashrc) -----
function loadEnv() {
  const f = path.join(DIR, ".env");
  if (!fs.existsSync(f)) return;
  for (const line of fs.readFileSync(f, "utf8").split(/\r?\n/)) {
    const m = line.match(/^\s*([A-Z_]+)\s*=\s*(.*)\s*$/);
    if (m && !process.env[m[1]]) process.env[m[1]] = m[2].replace(/^["']|["']$/g,
  }
}
loadEnv();

const WEBHOOK = (process.env.DISCORD_WEBHOOK_URL || "").trim();
const CLIENT_ID = (process.env.EBAY_CLIENT_ID || "").trim();
const CLIENT_SECRET = (process.env.EBAY_CLIENT_SECRET || "").trim();
const API = process.env.EBAY_API_BASE || "https://api.ebay.com";

const config = JSON.parse(fs.readFileSync(path.join(DIR, "config.json"), "utf8"))
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
const log = (...a) => console.log(new Date().toLocaleTimeString("fr-FR"), ...a);

// ---------- Mémoire des annonces déjà vues (pas de doublons) ----------
let seen = {};
try { seen = JSON.parse(fs.readFileSync(SEEN_FILE, "utf8")); } catch {}
function saveSeen() {
  const keys = Object.keys(seen);
  if (keys.length > 20000) for (const k of keys.slice(0, keys.length - 15000)) de
  fs.mkdirSync(path.dirname(SEEN_FILE), { recursive: true });
  fs.writeFileSync(SEEN_FILE, JSON.stringify(seen));
}

// ---------- Discord ----------
async function discord(payload) {
```

```
  for (let i = 0; i < 3; i++) {
    const r = await fetch(WEBHOOK, { method: "POST", headers: { "Content-Type": "
    if (r.status === 429) { await sleep(((await r.json().catch(() => ({}))).retry
    if (!r.ok) log("Discord erreur", r.status, await r.text().catch(() => ""));
    return;
  }
}

// ---------- eBay : jeton OAuth (renouvelé tout seul) ----------
let token = null, tokenExp = 0;
async function getToken() {
  if (token && Date.now() < tokenExp - 60_000) return token;
  const r = await fetch(`${API}/identity/v1/oauth2/token`, {
    method: "POST",
    headers: {
      "Content-Type": "application/x-www-form-urlencoded",
      Authorization: "Basic " + Buffer.from(`${CLIENT_ID}:${CLIENT_SECRET}`).toSt
    },
    body: "grant_type=client_credentials&scope=" + encodeURIComponent("https://ap
  });
  if (!r.ok) throw new Error(`Clés eBay refusées (${r.status}) : ${await r.text()
  const j = await r.json();
  token = j.access_token; tokenExp = Date.now() + j.expires_in * 1000;
  return token;
}

let callsToday = 0, day = new Date().toDateString();
async function ebaySearch(s, marketplace) {
  if (new Date().toDateString() !== day) { day = new Date().toDateString(); calls
  const filters = [
    `price:[${s.min ?? config.defaultMin}..${s.max}]`,
    "priceCurrency:EUR",
    "buyingOptions:{FIXED_PRICE|BEST_OFFER}", // pas d'enchères (prix trompeurs)
    `deliveryCountry:${config.deliveryCountry}`,
  ];
  if (!s.parts) filters.push("conditionIds:{1000|1500|1750|2000|2010|2020|2030|25
  const q = new URLSearchParams({ q: s.query, sort: "newlyListed", limit: "50", f
  if (s.category) q.set("category_ids", String(s.category));
  callsToday++;
  const r = await fetch(`${API}/buy/browse/v1/item_summary/search?${q}`, {
    headers: {
      Authorization: `Bearer ${await getToken()}`,
      "X-EBAY-C-MARKETPLACE-ID": marketplace,
      "X-EBAY-C-ENDUSERCTX": `contextualLocation=country=${config.deliveryCountry
      "Accept-Language": "fr-FR",
    },
```

```
  });
  if (r.status === 401) { token = null; return []; }
  if (r.status === 429) throw new Error("QUOTA");
  if (!r.ok) { log(`eBay ${r.status} sur "${s.query}"`); return []; }
  return (await r.json()).itemSummaries || [];
}

// ---------- Filtre titre (même logique que le 14 octobre) ----------
const norm = (t) => " " + t.toLowerCase().normalize("NFD").replace(/[̀-ͯ]/g, "").re
// "13" trouve "iPhone13", "660s" trouve "HD660S", mais "13" ne trouve pas "130"
const reCache = new Map();
function has(title, w) {
  if (!reCache.has(w)) {
    const k = norm(w).trim();
    const edge = (c) => (/\d/.test(c) ? "\\d" : "a-z");
    const body = k.split(" ").map((x) => x.replace(/[.*+?^${}()|[\]\\]/g, "\\$&")
    // code modèle qui finit par une lettre (660s, a7s) : on accepte la suite col
    const last = k[k.length - 1];
    const right = /[a-z]/.test(last) && /\d/.test(k) ? "" : `(?![${edge(last)}])`
    reCache.set(w, new RegExp(`(?<![${edge(k[0])}])${body}${right}`));
  }
  return reCache.get(w).test(title);
}
function ok(item, s) {
  const t = norm(item.title || "");
  if ((s.must || []).some((w) => !has(t, w))) return false;
  if ((s.not || []).some((w) => has(t, w))) return false;
  if (config.exclude.some((w) => has(t, w))) return false;
  return true;
}

function price(item) {
  const p = parseFloat(item.price?.value || 0);
  const ship = item.shippingOptions?.[0]?.shippingCost;
  const sp = ship ? parseFloat(ship.value || 0) : null;
  return { p, ship: sp, total: sp == null ? p : p + sp };
}

async function alert(item, s, marketplace) {
  const { p, ship, total } = price(item);
  const fb = item.seller?.feedbackPercentage, fbn = item.seller?.feedbackScore;
  const where = item.itemLocation?.country || "?";
  await discord({
    username: "ABYSS eBay Scanner",
    embeds: [{
      title: item.title.slice(0, 250),
```

```
      url: item.itemWebUrl,
      color: total <= s.max * 0.6 ? 0xff2d55 : 0x00c853, // rouge = très en desso
      thumbnail: item.image?.imageUrl ? { url: item.image.imageUrl } : undefined,
      fields: [
        { name: "💶 Prix", value: `**${p.toFixed(0)} €**` + (ship == null ? "" :
        { name: "🎯 Ton max", value: `${s.max} €`, inline: true },
        { name: "📦 État", value: item.condition || "?", inline: true },
        { name: "👤 Vendeur", value: fb ? `${fb} % (${fbn})` : "?", inline: true
        { name: "🌍 Pays", value: `${where} · ${marketplace.replace("EBAY_", "eBa
        { name: "🔎 Recherche", value: s.name || s.query, inline: true },
      ],
      footer: { text: "ABYSS eBay Scanner" },
      timestamp: new Date().toISOString(),
    }],
  });
}

// ---------- Boucle ----------
async function scanOne(s, marketplace, firstPass) {
  const items = await ebaySearch(s, marketplace);
  let sent = 0;
  for (const it of items) {
    const id = it.legacyItemId || it.itemId;
    if (seen[id]) continue;
    seen[id] = Date.now();
    if (firstPass) continue; // 1er tour : on mémorise sans spammer
    if (!ok(it, s)) continue;
    if (price(it).p > s.max) continue;
    await alert(it, s, marketplace);
    sent++;
    await sleep(400);
  }
  return sent;
}

async function main() {
  if (!WEBHOOK.startsWith("https://")) return console.log("❌ DISCORD_WEBHOOK_URI
  if (process.argv.includes("--test")) {
    await discord({ content: "✅ Test OK — ABYSS eBay Scanner peut écrire dans ce
    return console.log("Message test envoyé sur Discord.");
  }
  if (!CLIENT_ID || !CLIENT_SECRET) return console.log("❌ EBAY_CLIENT_ID / EBAY_
  await getToken();

  const searches = config.searches.filter((s) => s.active !== false);
  const markets = config.marketplaces;
```

```
  const callsPerRound = searches.length * markets.length;
  // Quota gratuit eBay ≈ 5000 appels/jour → on règle la vitesse tout seul pour n
  const pause = Math.max(config.minPauseSeconds * 1000, Math.ceil((86_400_000 / c
  log(`Scanner démarré : ${searches.length} recherches × ${markets.length} eBay ·
  await discord({ content: `🟢 **ABYSS eBay Scanner est en ligne** — ${searches.l

  let firstPass = Object.keys(seen).length === 0;
  for (;;) {
    let total = 0;
    for (const s of searches) {
      for (const m of markets) {
        try { total += await scanOne(s, m, firstPass); }
        catch (e) {
          if (e.message === "QUOTA") { log("Quota eBay du jour atteint — pause 30
          else log("Erreur :", e.message);
        }
        await sleep(pause);
      }
    }
    saveSeen();
    log(firstPass ? "1er tour fini : annonces existantes mémorisées, les alertes
    firstPass = false;
  }
}

process.on("SIGINT", () => { saveSeen(); process.exit(0); });
main().catch((e) => { console.error("❌", e.message); process.exit(1); });
```